

Motion Detector

Bezpečnostní systém s detekcí pohybu a vizualizací dat

Autor	Tomáš Voborný
Studijní obor	Informační technologie
Škola	Střední průmyslová škola
Školní rok	2025/2026
Jazyk	C# / C++ (Arduino)
Platforma	Windows 10/11

Dokumentace k maturitnímu projektu

1. Anotace

Motion Detector je bezpečnostní systém detekující pohyb pomocí PIR senzoru připojeného k Arduino Uno. Data jsou přenášena přes USB Serial port do desktopové aplikace napsané v C# s WPF rozhraním, která zobrazuje události v reálném čase, ukládá log do souboru a vykresluje graf detekcí. Celý systém je distribuován jako jeden spustitelný soubor bez nutnosti instalace.

Annotation (EN): Motion Detector is a security system that detects movement using a PIR sensor connected to an Arduino Uno. Data is transmitted via USB Serial to a C# WPF desktop application providing real-time event logging, motion charts, and audio alerts. The system is packaged as a single self-contained Windows executable.

2. Úvod

Projekt Motion Detector vznikl jako odpověď na potřebu jednoduchého, levného a snadno rozšiřitelného bezpečnostního systému. Komerční řešení jsou často drahá, proprietární a neumožňují uživateli plnou kontrolu nad systémem. Cílem bylo vytvořit kompletní end-to-end systém od fyzického senzoru až po uživatelsky přívětivou desktopovou aplikaci s vizualizací.

Systém se skládá ze dvou hlavních částí. První část je hardwarová — Arduino Uno s PIR senzorem HC-SR501 a LED diodou. Arduino zpracovává signál ze senzoru a odesílá strukturovaná data přes USB Serial port. Druhá část je softwarová — desktopová aplikace v C# s WPF rozhraním, která přijímá data, zobrazuje je v reálném čase a ukládá do log souboru.

Komunikace mezi Arduinem a aplikací probíhá přes sériový port rychlostí 9600 baud. Arduino odesílá zprávy ve formátu `MOTION;{timestamp}` při detekci pohybu a `CLEAR;{timestamp}` při ukončení pohybu. Aplikace tyto zprávy parsuje a aktualizuje UI.

Výsledná aplikace nabízí přehledný dark mode dashboard s tabulkou logů, grafem pohybů v čase, počítadlem detekcí a stavovým indikátorem senzoru. Vše je zabaleno do jednoho .exe souboru pomocí .NET self-contained publish, takže aplikaci lze spustit na libovolném Windows 10/11 počítači bez instalace .NET runtime.

3. Ekonomická rozvaha

3.1 Analýza konkurence

Na trhu existuje řada komerčních řešení pro detekci pohybu:

Produkt	Cena	Výhody	Nevýhody
Philips Hue Motion	1 500 Kč	Snadná instalace	Proprietární ekosystém
Aqara Motion Sensor	800 Kč	Zigbee integrace	Nutný hub, cloud
Ring Alarm	3 000 Kč+	Profesionální	Předplatné, cloud

Motion Detector (tento projekt)	~300 Kč	Open, bez cloudu, vizualizace	Pouze Windows
--	---------	-------------------------------	---------------

3.2 Unikátnost řešení

- Plná kontrola — žádný cloud, žádné předplatné, data zůstávají lokálně
- Vizualizace dat — live graf pohybů v čase, který komerční senzory nenabízejí
- Rozšiřitelnost — open source kód, snadno rozšiřitelné o další senzory
- Nízké náklady — Arduino Uno ~250 Kč, PIR senzor ~30 Kč
- Single .exe — žádná instalace, spustitelné okamžitě

3.3 Způsob propagace

- GitHub — zdrojový kód a dokumentace veřejně dostupné
- Školní prezentace — plakát formátu A4 a živá demonstrace
- Komunity — sdílení na Arduino fórech a C# komunitách

3.4 Náklady a návratnost investic

Celkové náklady na hardware:

- Arduino Uno: ~250 Kč
- PIR senzor HC-SR501: ~30 Kč
- LED dioda + odpor + jumper kabely: ~20 Kč
- Celkem: ~300 Kč

Softwarové náklady jsou nulové — použity výhradně bezplatné nástroje (.NET, JetBrains Rider Community, Arduino IDE). Projekt má vzdělávací charakter, návratnost investic je v získaných dovednostech a jako základ pro komerční rozšíření.

4. Vývoj

4.1 Použité technologie

Technologie	Verze	Účel
Arduino IDE	2.x	Vývoj firmware pro Arduino Uno
C++ (Arduino)	-	Programovací jazyk firmware
C# / .NET 8	8.0	Backend logika desktopové aplikace
WPF + XAML	.NET 8	Uživatelské rozhraní (Windows)
System.IO.Ports	9.0.0	Komunikace přes Serial port
LiveCharts2	2.0-rc6	Vizualizace grafu pohybů
MVVM pattern	-	Architektura aplikace

JetBrains Rider	2025.x	IDE pro C# vývoj
-----------------	--------	------------------

4.2 Architektura aplikace

Aplikace je členěna podle MVVM (Model-View-ViewModel) vzoru:

- — **datový model události (timestamp, typ)** Models/MotionEvent.cs
- — **kommunikace s Arduinem přes Serial port** Services/SerialService.cs
- — **ukládání událostí do log souboru** Services/LogService.cs
- — **business logika, binding na UI, graf** ViewModels/MainViewModel.cs
- — **WPF uživatelské rozhraní** MainWindow.xaml

Komunikační protokol Arduino → Aplikace:

```
MOTION;{millis}    // pohyb detekován, millis = uptime Arduina v ms
CLEAR;{millis}     // pohyb ukončen
```

4.3 Firmware (Arduino)

Klíčová část Arduino kódu používá stavový automat pro detekci hran (LOW→HIGH a HIGH→LOW), aby nedocházelo k opakovanému odesílání zpráv:

```
int pirPin = 2;
int ledPin = 8;
bool posledniStav = false;

void loop() {
    bool pohyb = digitalRead(pirPin);
    if (pohyb && !posledniStav) {
        Serial.println("MOTION;" + String(millis()));
        digitalWrite(ledPin, HIGH);
        posledniStav = true;
    } else if (!pohyb && posledniStav) {
        Serial.println("CLEAR;" + String(millis()));
        digitalWrite(ledPin, LOW);
        posledniStav = false;
    }
}
```

PIR senzor potřebuje 30 sekund kalibrační dobu po zapnutí, proto je v setup() vložen delay(30000).

4.4 Průběh vývoje

1. Návrh architektury a výběr komponent (Arduino Uno, PIR HC-SR501)
2. Implementace Arduino firmware — detekce stavových přechodů
3. Ladění PIR senzoru — nastavení citlivosti a time delay potenciometru
4. Vytvoření WPF projektu v JetBrains Rider s MVVM strukturou

5. Implementace SerialService — čtení dat z COM portu
6. Implementace LogService — zápis do motion_log.txt
7. Vytvoření UI v XAML — dark mode dashboard
8. Integrace LiveCharts2 — živý graf pohybů
9. Ladění COM port detection přes Windows Registry
10. Publish jako single self-contained .exe (dotnet publish)

5. Testování

Testování probíhalo manuálně v průběhu vývoje. Níže jsou popsány hlavní testovací scénáře:

#	Scénář	Postup	Očekávaný výsledek	Skutečný výsledek	Stav
T1	Detekce pohybu	Zamávat rukou před PIR senzorem ve vzdálenosti 1m	Aplikace zobrazí MOTION v logu, LED se rozsvítí, zazní zvuk, počítadlo +1	Vše proběhlo dle očekávání	PASS
T2	Ukončení pohybu	Přestat se pohybovat, počkat na konec detekce	Aplikace zobrazí CLEAR v logu, LED zhasne	Vše proběhlo dle očekávání	PASS
T3	Graf pohybů	Provést 5 pohybů po sobě	Graf zobrazí 5 bodů s časovými razítky	Graf se aktualizuje správně	PASS
T4	Log soubor	Po 3 detekcích zkontrolovat motion_log.txt	Soubor obsahuje záznamy ve formátu DD.MM.YYYY HH:mm:ss TYP	Záznamy jsou uloženy správně	PASS
T5	Nasazení aplikace	Zkopírovat .exe na jiný PC, spustit bez instalace .NET	Aplikace se spustí, zobrazí se UI, lze vybrat COM port	Aplikace funguje bez instalace	PASS

6. Nasazení a spuštění

6.1 Požadavky

- Windows 10 nebo Windows 11 (64-bit)
- Arduino Uno s nahraným firmware
- PIR senzor HC-SR501 zapojený na pin D2
- LED dioda zapojená na pin D8 (přes 220Ω odpor)
- USB kabel (Arduino → PC)

6.2 Zapojení hardware

Komponenta	Pin komponenty	Pin Arduino
PIR HC-SR501	VCC (červený)	5V
PIR HC-SR501	GND (hnědý)	GND

PIR HC-SR501	OUT (žlutý)	D2
LED dioda	Anoda (+)	D8
LED dioda	Katoda (-)	GND (přes 220Ω)

6.3 Postup spuštění

11. Nahrát firmware do Arduina přes Arduino IDE (soubor motion_detector.ino)
12. Připojit Arduino k PC přes USB kabel
13. Spustit MotionDetector.exe
14. Kliknout na tlačítko Obnovit pro načtení dostupných COM portů
15. Vybrat správný COM port (např. COM5) a kliknout Připojit
16. Zelený indikátor potvrdí úspěšné připojení
17. Zamávat před senzorem — v aplikaci se zobrazí detekce

6.4 Sestavení ze zdrojového kódu

Pro sestavení je potřeba .NET 8 SDK a JetBrains Rider nebo Visual Studio 2022.

```
git clone https://github.com/voborny/motion-detector
cd MotionDetector
dotnet restore
dotnet publish -c Release -r win-x64 --self-contained true -
p:PublishSingleFile=true
```

7. Licence

Projekt je uvolněn pod licencí MIT License.

MIT License

Copyright (c) 2026 Tomas Voborny

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

8. Odkaz na repozitář

Zdrojový kód, dokumentace a firmware jsou dostupné na GitHubu:

GitHub: <https://github.com/voborny/motion-detector>

Repozitář obsahuje:

- motion_detector.ino — Arduino firmware
- MotionDetector/ — zdrojový kód C# WPF aplikace
- docs/ — tato dokumentace
- README.md — stručný průvodce spuštěním

9. Závěr

Projekt Motion Detector splnil všechny stanovené cíle. Byl vytvořen funkční bezpečnostní systém zahrnující hardwarovou část (Arduino + PIR senzor) i softwarovou část (C# WPF aplikace) s obousměrnou komunikací přes Serial port.

Největší výzvou bylo řešení problémů s detekcí COM portů na různých systémech — výsledně bylo nutné přejít na čtení přímo z Windows registry místo standardního `System.IO.Ports.SerialPort.GetPortNames()`. Další výzvou byla kompatibilita knihovny LiveCharts2 s .NET 9, která si vyžádala přechod na .NET 8.

Projekt přinesl cenné zkušenosti v oblasti embedded programování (Arduino C++), desktop vývoje (C# WPF, MVVM), sériové komunikace a správy NuGet balíčků. Kód je strukturován přehledně a je připraven pro rozšíření — například o podporu více senzorů, email notifikace nebo webové rozhraní.

Celkově hodnotím projekt jako úspěšný. Výsledná aplikace je stabilní, uživatelsky přívětivá a řeší reálný problém za zlomek ceny komerčních alternativ.